

Reviewer Pack — Minimal Order of Quiver Representations and DPLL Proof Path ...

Entry 65a2033dbbb8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 19:03:56 UTC

Minimal Order of Quiver Representations and DPLL Proof Path Length Inequality

Entry ID: 65a2033dbbb8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 19:03:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quiver Theory) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every CNF formula φ with n variables, the minimal order of the quiver representation associated with φ is linearly correlated with its DPLL proof path length $l(\varphi)$, such that $\log(\min_order(Q(\varphi))) = \Theta(l(\varphi))$.

Rationale (proposer's reasoning):

Quiver representations encode graphical structures that can be used to represent the logical structure of a CNF formula. The minimal order provides an invariant that measures the complexity of the quiver itself, which could reflect the underlying complexity of the DPLL proof process.

Taxonomy category: Quiver Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4864733360ec2eee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $\log(\min_order(Q(\varphi)))$ and $l(\varphi)$ is ≥ 0.7 across at least 25 of the 30 random seeds, with no seed showing a correlation coefficient < 0.6 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quiver representation" AND "DPLL proof complexity" - "minimal order of quiver representations" AND CNF formula" - "Boolean satisfiability" AND "algebraic geometry quiver theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations, product

def generate_cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = [random.choice(variables) * (-1 if random.randint(0, 1) else 1)]
        while len(clause) < 3 and any(x == y or x == -y for x in clause):
            new_var = random.choice(variables)
            if new_var not in clause:
                clause.append(new_var * (-1 if random.randint(0, 1) else 1))
        clauses.append(clause)
    return clauses

def dpll_solve(cnf):
    def is_satisfiable(vars):
        for clause in cnf:
            if all(var not in vars and -var not in vars for var in clause):
                return False
        return True

    def backtrack(vars, assignment):
        if len(assignment) == n:
            return is_satisfiable(assignment)
        var = list(set(range(1, n + 1)) - set(assignment.keys()))[0]
        for val in [True, False]:
            assignment[var] = val
            if backtrack(vars, assignment):
                return True
            del assignment[var]
        return False
```

```

assignment = {}
return backtrack(variables, assignment)

def quiver_representation(cnf):
    n = len(cnf)
    Q = [[0] * n for _ in range(n)]
    for clause in cnf:
        for var1 in clause:
            for var2 in clause:
                if var1 != var2:
                    Q[abs(var1) - 1][abs(var2) - 1] += 1
    return Q

def min_order(Q):
    n = len(Q)
    visited = [False] * n
    order = 0

    def dfs(v):
        nonlocal order
        stack = [v]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                for v in range(n):
                    if Q[u][v] > 0 and not visited[v]:
                        stack.append(v)
                order += 1

    dfs(0)
    return order

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = 2 * n # Ensure at least one clause per variable
        cnf = generate_cnf(n, m)
        Q = quiver_representation(cnf)
        min_order_Q = min_order(Q)
        l_phi = dpll_solve(cnf)

        if l_phi is None:
            continue

        results.append({
            "n": n,
            "min_order_Q": min_order_Q,
            "l_phi": l_phi
        })

    if not results:

```

```

    return {
        "metric_name": "log(min_order(Q( $\phi$ ))) vs. DPLL proof path length",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No valid CNF formulas generated"
    }

log_min_order_Q = [Fraction(math.log(result["min_order_Q"]), math.e) for result in results]
l_phi_values = [result["l_phi"] for result in results]

correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(log_min_order_Q, l_phi_values)) / (math.sqrt(sum((x - mean_x) ** 2 for x in log_min_order_Q)) * math.sqrt(sum((y - mean_y) ** 2 for y in l_phi_values)))

mean_metric_value = correlation_coefficient
std_metric_value = 0.0
conjecture_holds = all(0.6 <= corr < 0.7 for corr in log_min_order_Q)
counterexample = "" if conjecture_holds else "Correlation coefficient outside [0.6, 0.7] range"

return {
    "metric_name": "log(min_order(Q( $\phi$ ))) vs. DPLL proof path length",
    "metric_value": mean_metric_value,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 187, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 527, 539, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 623, 629, 631, 637, 641, 643, 647, 653, 659, 661, 667, 671, 673, 677, 683, 687, 691, 693, 697, 701, 703, 707, 709, 713, 719, 727, 731, 733, 737, 739, 743, 749, 751, 757, 761, 763, 767, 769, 773, 779, 781, 787, 791, 793, 797, 803, 809, 811, 817, 821, 823, 827, 829, 833, 837, 839, 843, 847, 851, 853, 857, 859, 863, 867, 869, 873, 877, 881, 883, 887, 893, 897, 901, 903, 907, 911, 913, 917, 919, 923, 927, 929, 931, 933, 937, 939, 943, 947, 949, 953, 959, 961, 967, 971, 973, 977, 983, 989, 991, 993, 997, 1000]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "metric_value" in trial_result and trial_result["metric_value"] is not None:
            results.append(trial_result["metric_value"])

    mean_metric_value = sum(results) / len(results) if results else 0
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in results)) / len(results)
    support_fraction = sum(1 for result in results if 0.6 <= result < 0.7) / len(results)

    if all(0.6 <= result < 0.7 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(result < 0.6 or result >= 0.7 for result in results):
        first_failing_seed = seeds[next(i for i, result in enumerate(results) if not (0.6 <= result < 0.7))]
        print(f"RESULT: FALSIFIED counterexample='Correlation coefficient outside [0.6, 0.7] range'")
    else:
        print("RESULT: INCONCLUSIVE Reason=No valid CNF formulas generated")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10024559.py", line 138, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10024559.py", line 87, in run_trial
    cnf = generate_cnf(n, m)
         ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10024559.py", line 22, in generate_cnf
    while len clause < 3 and any(x == y or x == -y for x in clause):
                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10024559.py", line 22, in <genexpr>
    while len clause < 3 and any(x == y or x == -y for x in clause):
                               ^
```

NameError: name 'y' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient required to support the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15179
2	preregistration	ollama_remote	glm4:latest	0	0	10106
3	novelty	ollama_remote	glm4:latest	0	0	13256
4	novelty	ollama_remote	glm4:latest	0	0	8900
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55154
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18354
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17427
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17155
9	judge	ollama_remote	glm4:latest	0	0	28443

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 183975 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/65a2033dbbb8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/65a2033dbbb8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/65a2033dbbb8.tar.gz (if generated)